



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G2

System Integration and Architecture

System Design Document (SDD)

Project Title: MedBuddy

Prepared By: Aquino, Julianne Rose P.

Version: 1.6

Date: May 26, 2026

Status: Final

REVISION HISTORY TABLE

Versio n	Date	Author	Changes Made	Status
1.0	February 23, 2026	Aquino, Julianne Rose P.	Initial draft	Partial
1.1	March 2, 2026	Aquino, Julianne Rose P.	Added API specifications	Partial
1.2	March 2, 2026	Aquino, Julianne Rose P.	Updated database design	Partial
1.3	March 2, 2026	Aquino, Julianne Rose P.	Added UI/UX designs	Partial
1.4	March 2, 2026	Aquino, Julianne Rose P.	Incorporated feedback	Partial
1.5	March 2, 2026	Aquino, Julianne Rose P.	Final review and corrections	Partial
1.6	May 26, 2026	Aquino, Julianne Rose P.	Baseline version for development	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY.....	4
1.0 INTRODUCTION.....	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION.....	5
3.0 NON-FUNCTIONAL REQUIREMENTS.....	9
4.0 SYSTEM ARCHITECTURE.....	11
5.0 API CONTRACT & COMMUNICATION.....	12
6.0 DATABASE DESIGN.....	17
7.0 UI/UX DESIGN.....	18
8.0 PLAN.....	21

EXECUTIVE SUMMARY

1.1 Project Overview

MedBuddy is a multi-platform hospital appointment system built as a systems integration project. The system consists of three integrated components: a Spring Boot RESTful API backend, a React + Vite web application, and a Kotlin-based Android mobile application. All three components communicate through the Spring Boot backend and share a single PostgreSQL relational database hosted on Neon. The backend is deployed on Render.com and the web frontend on Vercel. The platform enables patients to register, book appointments, view their medical records, track billing, and submit feedback — all from either the web or mobile interface.

1.2 Objectives

1. Build a fully integrated hospital appointment system across three platforms: Spring Boot feature sliced backend API, React + Vite web frontend, and Kotlin Android mobile frontend (XML-based MVVM).
2. Implement JWT-based authentication with BCrypt password hashing, protected routes, a /me endpoint, and role-based access control for Patient and Doctor roles.
3. Implement Google OAuth2 login that generates a custom JWT upon successful third-party authentication.
4. Consume the OpenFDA Drug Label API and display drug information meaningfully within the system.
5. Support file upload (lab results, prescription PDFs) stored on Supabase Storage and linked to database records via signed URLs.
6. Send transactional emails via Brevo SMTP for account-related events and system notifications.
7. Deploy the backend to Render.com and web frontend to Vercel.com and connect the Android app via Retrofit to the same live API.

1.3 Scope

Included Features:

- User registration and login (email/password) with BCrypt password hashing and JWT issuance
- Google OAuth2 login that generates a custom JWT after successful authentication
- Role-based and payment-state-based access control with PATIENT and DOCTOR roles
- Protected routes on both React web and Android app (JWT in Authorization header)
- /me endpoint returning the currently authenticated user's profile
- Appointment booking as the core business module with full CRUD operations and validation
- Automated 30-minute slot generation (nightly job, 14 days ahead) with weekly template and date override support
- Medical record access (diagnoses and prescriptions including medicine name, dosage, route, frequency, duration) per completed appointment
- File upload support via Supabase Storage; signed URL retrieval for secure downloads
- Payment tracking with PayMongo sandbox integration (checkout sessions, webhooks, receipts)
- Rating and feedback submission per completed appointment
- OpenFDA Drug Label API integration displayed in an About Your Medication card
- Brevo SMTP email integration for welcome, appointment, reminder, feedback alert, and payment receipt emails
- Deployment: Spring Boot on Render, React on Vercel, PostgreSQL on Neon, Android APK via sideload

Excluded Features:

- Real-time chat or direct patient-to-staff messaging (out of scope for this iteration)
- Real payment gateway processing (payment tracking is implemented; live gateway is deferred)
- Push notifications (Firebase Cloud Messaging — deferred to future iteration)

- Telemedicine or video consultation
- Multi-hospital or multi-branch support
- Insurance or PhilHealth integration
- Advanced analytics or reporting dashboard

1.0 INTRODUCTION

1.1 Purpose

This document is the comprehensive design and integration specification for MedBuddy. It details system requirements, architecture, API design, database schema, security implementation, integration features, and deployment strategy. It serves as the primary reference for development and academic evaluation.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: MedBuddy

Domain: Healthcare / Hospital Appointment Management

Primary Users: Patients and Doctors

Problem Statement: Patients struggle with manual appointment scheduling, lack of access to their medical records, and opaque billing. Doctors have no centralized tool to manage their availability and patient records.

Solution: A multi-platform system (React web + Kotlin Android) backed by a Spring Boot REST API and PostgreSQL database, providing seamless appointment management, medical record access, transparent billing, and feedback collection across platforms.

2.2 Core User Journeys

Journey 1: Patient Registration and First Appointment

A new patient signs up, uploads prior records if available, and books an appointment with a reservation fee.

1. Patient opens the MedBuddy app or website.
2. Clicks Sign Up and enters basic personal details.
3. Receives a welcome or verification email.
4. Patient is logged into their account.
5. Browses doctors and views profiles and specializations.
6. Selects a preferred appointment date and time from available 30-minute slots.
7. Optionally uploads previous medical records or lab results for the doctor to review before the visit.
8. Enters the reason for the visit and proceeds to booking.
9. Pays a reservation fee to secure the appointment.
10. Appointment becomes confirmed and appears in the dashboard.

Journey 2: Returning Patient — View Records and Complete Payment

A returning patient reviews appointment details and completes payment to unlock records.

1. Patient logs into the app.
2. Opens My Appointments to view upcoming and completed visits.
3. Selects a completed appointment.
4. Sees that medical records and files are locked pending payment.
5. Opens the payment section and views the remaining balance.
6. Completes the payment.
7. Receives a payment confirmation and receipt.
8. Medical records, prescriptions, and uploaded files become fully accessible.
9. Patient downloads or views medical documents.
10. Patient rates the appointment and optionally leaves feedback.

Journey 3: Sign In with Google

1. A patient or doctor logs in using their Google account.
2. User clicks Continue with Google.
3. Signs in using their Google account.
4. Grants permission to share basic profile details.
5. The system checks if an account already exists.
6. If new, an account is created automatically.
7. If existing, the user is logged in immediately.
8. User enters the app and continues normally.

Journey 4: Doctor Manages Schedule and Completes Appointment

A doctor prepares for consultation, reviews patient uploads, and records medical results.

1. Doctor logs into the system.
2. Sets weekly availability templates by day of week.
3. System nightly job generates 30-minute slots 14 days ahead; template saves also trigger async slot rebuild.
4. Views confirmed patients for the day.
5. Opens a patient's appointment details.
6. Reviews any medical files uploaded earlier by the patient.
7. Conducts the consultation.
8. Marks the appointment as completed.
9. Records diagnosis and prescriptions.
10. Uploads lab results or supporting medical files if needed.
11. The system generates the final bill, deducting the reservation fee, and notifies the patient that payment is required to unlock records.

Journey 5: Medical File Upload and Access Control

Both patient and doctor upload files, but access depends on payment completion.

Before Appointment (Patient Upload)

1. Patient uploads previous medical records while booking or before the visit.
2. Files become available for the doctor to review during consultation.

After Appointment (Doctor Upload)

3. Doctor uploads lab results or medical documents to the appointment record.

Access Rules

4. Patient can see that records exist but cannot open or download them yet.
5. Medical records and files remain locked while payment is pending.
6. Patient completes remaining payment.
7. All medical records and uploaded files become accessible.
8. Patient downloads or views files on their device.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout)
2. Appointment booking — create, view, update, cancel
3. Automated slot generation with weekly templates and date-specific overrides
4. Medical records access (view diagnoses, prescriptions with full drug fields)
5. Role-based and state-based access control (PATIENT, DOCTOR, payment-gated records)
6. PayMongo sandbox integration with webhook-driven payment settlement

SHOULD HAVE

1. Doctor availability calendar with real-time slot filtering
2. Queue status viewing (polling-based, auto-refresh)
3. Payment tracking per appointment (fee visibility, status)
4. Rating and feedback submission per completed appointment
5. Basic input validation feedback on all forms

6. Responsive design for all screen sizes (web)
7. Search and filter doctors by specialization

COULD HAVE

1. Appointment reminder emails (scheduled 24 hours before)
2. Doctor average rating displayed on doctor profile
3. About Your Medication widget from OpenFDA on dashboard

WON'T HAVE

1. Real-time patient-to-staff chat or messaging
2. Live payment gateway (production mode)
3. Push notifications (Firebase Cloud Messaging)
4. Telemedicine or video consultation
5. Multi-hospital or multi-branch management
6. Insurance or PhilHealth integration
7. Advanced analytics or reporting dashboard

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Google OAuth2 Login, My Profile
- **Fields:** First Name, Last Name, Email, Password, Confirm Password, Role (Patient/Doctor)
- **Validation:** Email format, password minimum 8 characters, password match, unique email, role required
- **API Endpoints:** POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me, GET /api/auth/health, GET /api/auth/oauth2/google (redirect init), POST /api/auth/oauth2/google/token
- **Security:** JWT (HS256) generated by backend for both password and OAuth login; BCrypt password hashing; Spring Security filter chain; Google OAuth2 browser-redirect flow with custom JWT issuance; token stored in localStorage (web) and EncryptedSharedPreferences (Android)

Feature: Appointment Booking (Core Business Module)

- Screens: Appointment List, Book Appointment, Appointment Detail, Queue Status
- Fields: Doctor (select), Date (from slot calendar), Time Slot (from generated slots), Reason for Visit
- Validation: Date not in the past, one-day-ahead booking rule, slot must be AVAILABLE, doctor not double-booked, patient must be authenticated
- Status Transitions: BOOKED → COMPLETED (doctor only) or BOOKED → CANCELED (either party)
- API Endpoints: GET /api/appointments, POST /api/appointments, GET /api/appointments/{id}, PUT /api/appointments/{id}, PATCH /api/appointments/{id}/cancel, PATCH /api/appointments/{id}/complete, GET /api/appointments/queue, GET /api/appointment-slots (separate slot resource)

Feature: Doctor Availability & Slot Management

- **Screens:** Availability Calendar (patient — read only), Manage Availability (doctor — CRUD)
- **Weekly templates:** define recurring hours by day of week via doctor_schedule_templates
- **Date-specific overrides:** doctor_availability records with AVAILABLE (custom hours) or UNAVAILABLE (blocks) behavior
- **Precedence rule:** UNAVAILABLE exception removes unbooked slots; AVAILABLE exception uses custom start/end; otherwise weekly template applies
- **Automated slot generation:** nightly job at 1:00 AM UTC generates 30-minute slots 14 days ahead for every doctor
- Template changes trigger async listener that rebuilds slots; exception save/delete updates affected date's unbooked slots directly
- **API Endpoints:** GET /api/doctors/{id}/availability, POST /api/doctors/{id}/availability, PUT /api/doctors/availability/{id}, DELETE /api/doctors/availability/{id}, GET /api/appointment-slots, plus /api/legacy/availability namespace

Feature: Medical Records

- **Screens:** Medical History (patient), Create/Edit Record (doctor), Record Detail with File Download
- **Fields:** Diagnosis, Medicine Name, Dosage, Route, Frequency, Duration, Prescription Notes, File Upload (optional PDF or image)
- **Access Control:** Patients read own records only after payment is PAID; doctors create and read for their own patients
- **External API:** Prescription drug name sent to OpenFDA Drug Label API; medication information displayed in About Your Medication section
- **File Storage:** Supabase Storage; files referenced via storage_path and retrieved via signed URLs
- **API Endpoints:** GET /api/medical-records, GET /api/medical-records/{id}, POST /api/medical-records, PUT /api/medical-records/{id}, POST /api/medical-record-files (signed file workflows), GET /api/medical-record-files/{id}, plus legacy /api/files and /api/record-files routes

Feature: Payment Tracking

- **Screens:** Payment Status (patient), Update Payment (doctor)
- **Fields:** Fee Amount, Payment Status, PayMongo Session ID, PayMongo Payment Intent ID, Transaction ID, Paid At timestamp
- **Status Values:** PENDING, PAID, FAILED, REFUNDED
- **Gateway:** PayMongo or PayPal sandbox — real provider, test mode; no simulated payments
- **API Endpoints:** POST /api/payments/initiate, POST /api/payments/confirm, POST /api/payments (direct creation), GET /api/payments/{id}, GET /api/payments/appointment/{appointmentId}, PUT /api/payments/{id}/total-bill (doctor-side update), POST /api/payments/webhook (public, webhook signature verified)

Feature: Rating and Feedback

- **Screens:** Submit Feedback (patient, post-completed appointment), Doctor Profile (average rating)
- **Fields:** Rating Score (1–5 stars), Feedback Comment (optional)
- **Constraints:** One feedback per appointment; only available after status = COMPLETED

- **API Endpoints:** exposed under both /api/ratings and /api/feedback — POST /api/ratings, GET /api/ratings/doctor/{id}, GET /api/ratings/average/{doctorId}, GET /api/ratings/patient/{patientId}, GET /api/ratings/appointment/{appointmentId}, DELETE /api/ratings/{id}

Feature: External API Integration

- **API:** OpenFDA Drug Label API — <https://api.fda.gov/drug/label.json> — free, no API key required, no sign-up
- **Integration Point:** Spring Boot DrugInfoService → RestTemplate → OpenFDA endpoint → parsed into DrugInfoDto; response is cached
- **Trigger:** Patient opens a Medical Record detail view; prescription drug name passed to Spring Boot; Spring Boot calls OpenFDA; returns drug info to client
- **Data Displayed:** Drug indications and usage, warnings, dosage instructions — shown in About Your Medication card on both web and Android
- **Error Handling:** If OpenFDA is unreachable or returns no results, graceful
- Also exposed via /api/medical-records drug-info endpoint for direct client queries

2.5 Acceptance Criteria

AC-01: Successful Patient Registration

Given I am a new user on the MedBuddy web or Android app

When I enter valid first name, last name, email, and a strong password, select role "Patient", and click Register

Then my account should be created with a BCrypt-hashed password

And a Patient profile record should be linked to my account

And a welcome/verification email should be sent to my email address

And I should be automatically logged in and receive a JWT

And I should be redirected to the patient dashboard

AC-02: Email/Password Login

Given I am a registered user

When I enter my correct email and password and click Login

Then Spring Boot should validate my credentials against the BCrypt hash

And return a JWT containing my user ID, email, and role

And I should be redirected to my role-appropriate dashboard

AC-03: Google OAuth2 Login

Given I click "Continue with Google" and grant permission

When Spring Boot verifies my Google identity

Then a user record should be created with provider = "GOOGLE" if I am new

And Spring Boot should issue its own custom JWT (not a Google token)

And I should land on the patient dashboard as a fully authenticated user

AC-04: Book an Appointment

Given I am logged in as a PATIENT

When I select a doctor, choose an available date and time slot, enter a reason, and confirm

Then the appointment should be created with status = BOOKED

And the selected availability slot should be marked UNAVAILABLE

And an appointment confirmation email should be sent to me

And the appointment should appear in both my list and the doctor's schedule

AC-05: Prevent Double Booking

Given I am logged in as a PATIENT

When I attempt to book a time slot already booked by another patient

Then the API should return HTTP 409 Conflict

And an error message should be displayed on web and Android

And no duplicate appointment should be created in the database

AC-06: Role-Based Access Control

Given I am logged in as a PATIENT

When I attempt to access a DOCTOR-only action such as creating a medical record
Then the API should return HTTP 403 Forbidden

And the UI should not display doctor-only controls to me

And no data should be modified in the database

AC-07: Doctor Marks Appointment Completed

Given I am logged in as a DOCTOR and viewing a BOOKED appointment

When I click "Mark as Completed"

Then the appointment status should be updated to COMPLETED

And a Payment record should be automatically created with status PENDING

And the patient should receive an email notification that their record is ready

AC-08: Upload and Download Medical File

Given I am logged in as a DOCTOR creating a medical record

When I upload a PDF lab result and save the record

Then the file should be stored on the server and its URL saved in the database

And the patient should see a "Download Lab Result" button on their record detail

And clicking it should download or open the file on both web and Android

AC-09: Payment Gateway — Success Flow

Given I am a PATIENT with a PENDING payment for a completed appointment

When I initiate and complete the test transaction in the sandbox payment UI

Then the payment gateway should send a webhook to Spring Boot confirming success
And the payment status should be updated to PAID in the database

And a payment receipt email should be sent to me

AC-10: External API Data Display

Given I am a PATIENT viewing a Medical Record detail that contains a prescription

When the record detail screen loads on web or Android

Then Spring Boot should call the OpenFDA Drug Label API using the prescribed drug name from the record

And an "About Your Medication" card should appear below my prescription details showing the drug's indications and usage, warnings, and dosage instructions

And if OpenFDA returns no results for the drug name, the message "Drug information currently unavailable" should be shown in place of the card

And if the OpenFDA API is unreachable or times out, the same fallback message should appear

And in both fallback cases the rest of the medical record — diagnosis, prescription details, and file download — should still load and display normally without any errors

AC-11: Slot Generation and Availability Precedence

Given a doctor has a weekly schedule template and date-specific exceptions

When the nightly job runs at 1:00 AM UTC

Then 30-minute slots should be generated for the next 14 days based on template hours

And UNAVAILABLE exceptions should remove all unbooked slots for that date

And AVAILABLE exceptions should override the template with custom start/end times

And template changes should trigger an immediate async slot rebuild for affected dates

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests (measured at Render.com; excludes external API calls)
- Web page load time: ≤ 3 seconds on broadband
- Android app cold start: ≤ 3 seconds (from tap to login screen fully rendered)
- Database queries: ≤ 500 ms per query (indexes on foreign keys and status columns)
- Concurrent users: up to 50 (scoped for academic/demo use; Render free tier limitation)
- File upload size limit: maximum 10MB per file, enforced by Spring Boot MultipartFile config
- External API timeout: maximum 5 seconds for OpenFDA Drug Label API calls; graceful fallback message shown on timeout — medical record view unaffected
- Slot generation job: nightly at 1:00 AM UTC; must complete within 60 seconds for all doctors

3.2 Security Requirements

- HTTPS for all communications (TLS certificates auto-managed by [Render](#) and Vercel)
- JWT authentication (HS256 signed; configurable expiration; validated on every request via OncePerRequestFilter)
- Backend issues its own JWT for both password login and successful Google OAuth2 — no Google token is forwarded to clients
- BCrypt password hashing (Spring Security PasswordEncoder; minimum cost factor 10)
- No plain-text passwords (password_hash column only; excluded from all DTO responses)
- Token storage: web app stores medbuddy_token and medbuddy_user in localStorage; Android stores token in EncryptedSharedPreferences with a separate encrypted payment session store

- SQL injection prevention (Spring Data JPA parameterized queries only; no raw SQL string concatenation)
- XSS protection (Spring Security default headers; React escapes output by default)
- CORS configured to allow only the Render.com React origin and Android; all other origins blocked
- Role enforcement via `@PreAuthorize` on all non-public endpoints; HTTP 403 returned for unauthorized roles
- State-based access control: patient access to medical records and signed file URLs is blocked until the related appointment payment status is PAID
- Rate limiting: 100 requests/minute per IP (Spring Boot filter)
- PayMongo webhook signature verification on every incoming webhook event
- File access control: download endpoint requires valid JWT; files not publicly browseable
- All secrets (DB password, JWT secret, SMTP credentials, API keys) stored as environment variables on Render — never in source code
- Session expiry broadcast on Android: `ACTION_SESSION_EXPIRED` broadcast sent when JWT is invalidated; returns user to LoginActivity

3.3 Compatibility Requirements

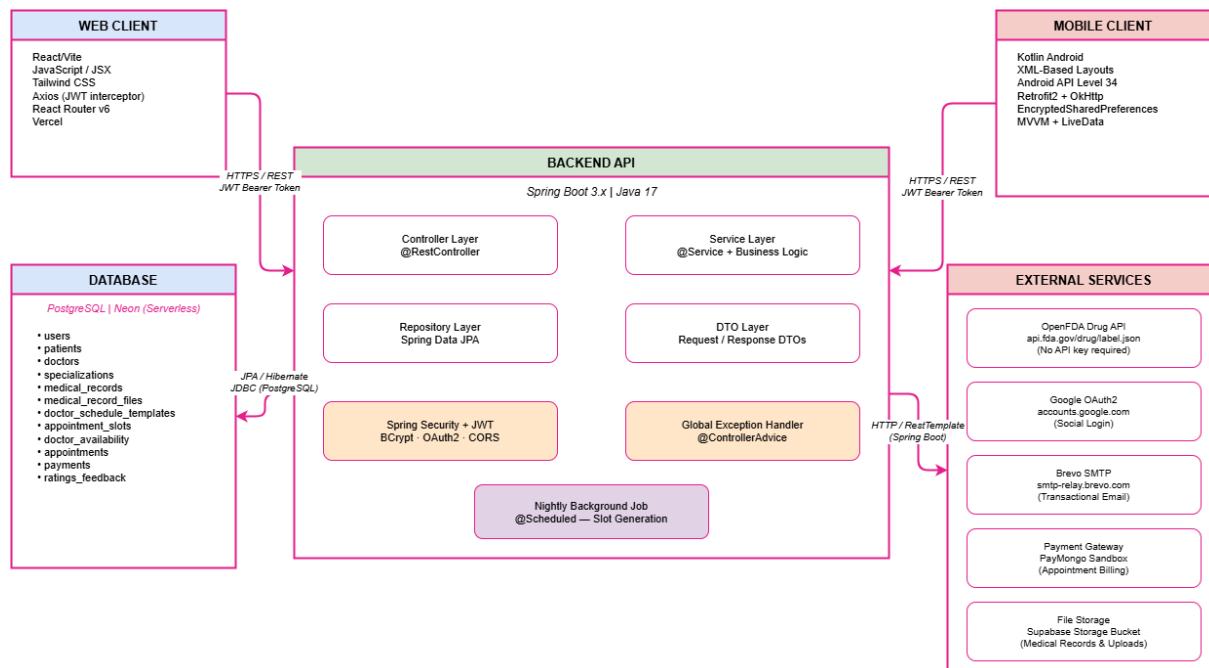
- Web Browsers: Chrome, Firefox, Safari, Edge (latest 2 major versions)
- Android: API Level 34 (Android 14) target SDK; minimum SDK API 26 (Android 8.0)
- Screen Sizes: Mobile (360px+), Tablet (768px+), Desktop (1024px+); responsive React layout
- Android screen sizes: standard phone sizes; minimum 360dp width; touch targets $\geq 48dp$
- Backend: Linux (Ubuntu) on Render.com (Web Service); Java 17+ runtime
- Web Frontend: React app deployed on Vercel (Static Site); auto-deploys from GitHub
- Database: PostgreSQL on Neon (free tier, serverless; connected to Spring Boot via JDBC URL environment variable)

3.4 Usability Requirements

- New patient completes registration and books first appointment within 5 minutes
- All validation errors display clear, field-specific messages with recovery guidance
- Consistent navigation structure across all pages on web; consistent bottom nav on Android
- All API calls show a loading indicator; no silent blank screens during data fetch
- Screens with no data show a helpful empty state message (e.g., "No appointments yet — book one now")
- Minimum 48x48dp touch targets for all interactive elements on Android
- Keyboard navigation support for all interactive elements on web
- WCAG 2.1 Level AA compliance targeted for web; sufficient color contrast ratios maintained

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



4.2 Architecture Overview

MedBuddy uses a feature-sliced monolithic backend served over HTTPS/REST. The web and Android clients both communicate exclusively through the Spring Boot API, which owns all business logic. No direct database access occurs from the clients.

4.3 Backend Architecture — Feature-Sliced Spring Boot

The backend is a Spring Boot 3.x feature-sliced codebase, not a classic layered MVC application.

Controllers, services, repositories, DTOs, and slice-specific logic are co-located under feature folders.

Shared infrastructure lives in a separate shared package.

Feature slices:

- auth — registration, login, logout, /me, Google OAuth2 flow
- user — user profile management, multipart profile-image upload
- doctor — doctor listing, profile, specialization lookup
- specialization — specialization CRUD
- appointment — booking, status transitions, queue
- schedule — weekly templates (doctor_schedule_templates) and date-specific exceptions
(doctor_availability); nightly slot generation job; async slot rebuild listeners
- payment — PayMongo checkout session creation, confirmation, webhook handler, total-bill
updates
- feedback — ratings submission, doctor average rating, appointment-level feedback lookup
- medicalrecords — record CRUD, drug info proxy to OpenFDA
- prescription — prescription field management (medicine_name, dosage, route, frequency,

duration)

- dashboard — doctor dashboard aggregate data

Shared package:

- config — SecurityConfig, CorsConfig, application properties
- filters — JwtAuthFilter (OncePerRequestFilter), rate-limit filter
- models — enums, base entity classes
- exceptions — GlobalExceptionHandler (@ControllerAdvice), custom exception types

4.4 Frontend Architecture — React + Vite

The web frontend is built with React 18 + Vite using JavaScript/JSX (no TypeScript).

State management

uses React Context API. Routing uses React Router v6. HTTP calls use Axios with a JWT interceptor.

Styling uses Tailwind CSS. The app is deployed as a static site on Vercel with auto-deploy from GitHub.

Key patterns:

- AuthContext: stores JWT and user profile; provides login/logout helpers
- ProtectedRoute: wraps role-restricted pages; redirects unauthenticated users to /login
- Axios instance: automatically attaches Authorization: Bearer <token> header

MedBuddy — System Design Document | Version 1.7

Page 17

- Google OAuth2: browser redirect to backend OAuth init route; token received via callback page

URL params

- Token storage: medbuddy_token and medbuddy_user in localStorage

4.5 Mobile Architecture — Android XML MVVM

The Android app is built with Kotlin using XML-based layouts and an MVVM pattern. Jetpack Compose is

not used. The app uses Activities, Fragments, ViewBinding, Retrofit 2 for network calls, and coroutines for

async operations.

Key patterns:

- MainActivity: checks stored JWT, reads user role from cached JSON

(EncryptedSharedPreferences), loads role-specific bottom navigation menu, swaps fragments

accordingly

- Session expiry: when JWT is invalidated the app sends an ACTION_SESSION_EXPIRED broadcast and returns user to LoginActivity

- Deep links: OAuthCallbackActivity handles medbuddy://oauth-callback; PaymentResultActivity

handles medbuddy://payment

- EncryptedSharedPreferences: stores JWT token and user profile; separate encrypted store for

payment session

- Retrofit 2 + OkHttp: JWT interceptor attaches token to every request; BASE_URL points to live Render API

Android screen inventory (full implementation):

- Auth: Login, Register, OAuthCallbackActivity

- Patient: Dashboard, Find Doctor (search + filter), Book Appointment, Appointment Details, My

Appointments, Medical History, Medical Record Detail, Billing, Feedback, Ratings, Profile Edit,

PaymentResultActivity

- Doctor: Dashboard, Appointment Management, Schedule Management, Patient Records, Record

Creation, Availability Management, Profile Edit

- Fragments: numerous patient and doctor fragments per section

Note: browser-only web concerns (protected routes, callback pages, loading overlays, unsaved-changes

guards) are labeled web-only and are not replicated in the Android app. The Android app independently

implements schedule management, medical records, file uploads, feedback, ratings, and payment result

handling.

Technology Stack:

Backend	Java 17, Spring Boot 3.x, Spring Security + JWT (HS256), Spring Data JPA (Hibernate), Spring Mail, OAuth2 Client, scheduled tasks (@Scheduled), async listeners (@EventListener)
Database	PostgreSQL on Neon (serverless, free tier); connected via JDBC URL environment variable
Web Frontend	React 18, JavaScript/JSX, Vite, Tailwind CSS, Axios, React Router v6, Context API
Mobile	Kotlin, XML-based layouts (Jetpack Compose NOT used), Activities + Fragments + ViewBinding, Retrofit 2, OkHttp, Coroutines, EncryptedSharedPreferences, Android API Level 34
File Storage	Supabase Storage — files uploaded with storage_path; retrieved via signed URLs
Email	Brevo (Sendinblue) SMTP — welcome,

	appointment, reminder, feedback alert, payment receipt emails
Payments	PayMongo sandbox — checkout session creation, webhook signature verification, payment intent confirmation
External API	OpenFDA Drug Label API (https://api.fda.gov/drug/label.json); cached DrugInfoDto
Deployment	Backend: Render.com (JAR via GitHub); Web: Vercel (static site); DB: Neon PostgreSQL
Build Tools	Maven (backend), npm/Vite (web), Gradle (Android)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

- **Base URL (local):** <http://localhost:8080/api>
- **Base URL (production):** <https://it342-medbuddy-aquino.onrender.com/>
- **Format:** JSON for all requests/responses
- **Authentication:** JWT Bearer token in Authorization: Bearer <token>
- All protected endpoints return HTTP 401 if token is missing/invalid; HTTP 403 if role is insufficient
- **Response Structure:**

```
json
```

```
{
```

```
  "success": boolean,
```

```
  "data": object|null,
```

```
  "error": {
```



```

    "code": string,
    "message": string,
    "details": object|null
  },
  "timestamp": string
}

```

5.2 Authentication Endpoints

Method	Endpoint	Description / Notes
POST	/api/auth/register	Register new user. Body: { email, password, role, firstName, lastName, phoneNumber?, specialization? (required if role=DOCTOR) }. Returns: { token, user }
POST	/api/auth/login	Email/password login. Body: { email, password }. Returns: { token, user }
GET	/api/auth/me	Returns currently authenticated user profile (requires JWT).
POST	/api/auth/logout	Invalidates session. Returns 200.
GET	/api/auth/health	Health check. Returns { status: UP }. No auth required.
GET	/api/auth/oauth2/google	Initiates Google OAuth2 browser-redirect flow. Backend stores redirect state in session.
POST	/api/auth/oauth2/google/token	Mobile token exchange — stub only, returns 501 Not Implemented.

5.3 User & Doctor Endpoints

Method	Endpoint	Description / Notes
GET	/api/users/me	Returns current user profile.
PUT	/api/users/profile	Update current user profile. Body: { firstName, lastName, phoneNumber }.
POST	/api/users/profile-image	Multipart profile image upload. Returns updated profileImageUrl.

GET	/api/users/doctors	List all doctors. Returns DoctorDto[].
GET	/api/doctors/{id}	Get single doctor profile including specializations.
GET	/api/doctors/{id}/availability	Get doctor availability slots for a date range.
POST	/api/doctors/{id}/availability	Doctor creates an availability override. Body: { date, startTime, endTime, status }.
PUT	/api/doctors/availability/{id}	Doctor updates an availability override.
DELETE	/api/doctors/availability/{id}	Doctor deletes an availability override; regenerates affected slots.

5.4 Appointment Endpoints

Method	Endpoint	Description / Notes
GET	/api/appointments	List appointments (filtered by role — patient sees own; doctor sees assigned).
POST	/api/appointments	Book appointment. Body: { doctorId, slotId, notes? }. One-day-ahead rule enforced.
GET	/api/appointments/{id}	Get appointment by ID.
PUT	/api/appointments/{id}	Update appointment details.
PATCH	/api/appointments/{id}/cancel	Cancel appointment. Available to patient (own) or doctor.
PATCH	/api/appointments/{id}/complete	Mark appointment completed. Doctor only. Auto-creates PENDING payment.
GET	/api/appointments/my	Alias: current user's appointments.
GET	/api/appointments/queue	Queue status for current patient (polling-based).
GET	/api/appointment-slots	List available 30-minute slots. Query params: doctorId, date.

5.5 Schedule Endpoints

Method	Endpoint	Description / Notes
GET	/api/schedule/templates	Get current doctor's weekly schedule templates. Doctor only.
POST	/api/schedule/templates	Save/replace weekly schedule template. Triggers async slot rebuild. Doctor only.
GET	/api/schedule/exceptions	Get date-specific exceptions for current doctor.

POST	/api/schedule/exceptions	Create date exception (AVAILABLE or UNAVAILABLE). Doctor only.
DELETE	/api/schedule/exceptions/{date}	Delete date exception; updates affected unbooked slots. Doctor only.
GET	/api/legacy/availability	Legacy availability endpoint; retained for backward compatibility.

5.6 Payment Endpoints

Method	Endpoint	Description / Notes
POST	/api/payments/initiate	Initiate PayMongo checkout session. Body: { appointmentId }. Returns checkout URL.
POST	/api/payments/confirm	Confirm payment intent after checkout. Body: { appointmentId, paymentIntentId }.
POST	/api/payments	Direct payment creation. Body: { appointmentId, feeAmount }. Doctor only.
GET	/api/payments/{id}	Get payment by payment ID.
GET	/api/payments/appointment/{appointmentId}	Get payment by appointment ID.
PUT	/api/payments/{id}/total-bill	Doctor updates total bill amount. Body: { totalBill }. Doctor only.
POST	/api/payments/webhook	Public PayMongo webhook. Verifies signature, settles payment, sends Brevo receipt.

5.7 Medical Record Endpoints

Method	Endpoint	Description / Notes
GET	/api/medical-records	List medical records for current user (role-filtered).
POST	/api/medical-records	Create medical record. Doctor only. Body: { appointmentId, diagnosis, medicineName, dosage, route, frequency, duration, prescriptionNotes? }.
GET	/api/medical-records/{id}	Get record. Patient access gated by payment=PAID.
PUT	/api/medical-records/{id}	Update record. Doctor only.
DELETE	/api/medical-records/{id}	Delete record. Doctor only.

GET	/api/medical-records/appointment/{appointmentId}	Get record by appointment.
GET	/api/medical-records/drug-info	Proxy to OpenFDA. Query param: drugName. Returns cached DrugInfoDto.
POST	/api/medical-record-files	Upload file to Supabase Storage. Returns storage_path and file_url.
GET	/api/medical-record-files/{id}/signed-url	Get signed Supabase URL for secure download. Patient access gated by payment=PAID.
GET	/api/files/{recordId}	Legacy file listing for record (retained for backward compatibility).
POST	/api/record-files	Legacy file upload route (retained for backward compatibility).

5.8 Feedback & Ratings Endpoints

Method	Endpoint	Description / Notes
POST	/api/ratings	Submit rating/feedback. PATIENT only. Body: { appointmentId, doctorId, ratingScore (1-5), feedbackComment? }. One per appointment.
GET	/api/ratings/doctor/{doctorId}	Get all ratings for a doctor (public).
GET	/api/ratings/average/{doctorId}	Get doctor average rating (public).
GET	/api/ratings/patient/{patientId}	Get ratings submitted by a patient (auth required).
GET	/api/ratings/appointment/{appointmentId}	Get rating for a specific appointment.
DELETE	/api/ratings/{id}	Delete rating. PATIENT owner only.
GET	/api/feedback/doctor/{doctorId}	Alias endpoint — same data as /api/ratings/doctor/{doctorId}.

6.0 Error Handling

HTTP Status Codes:

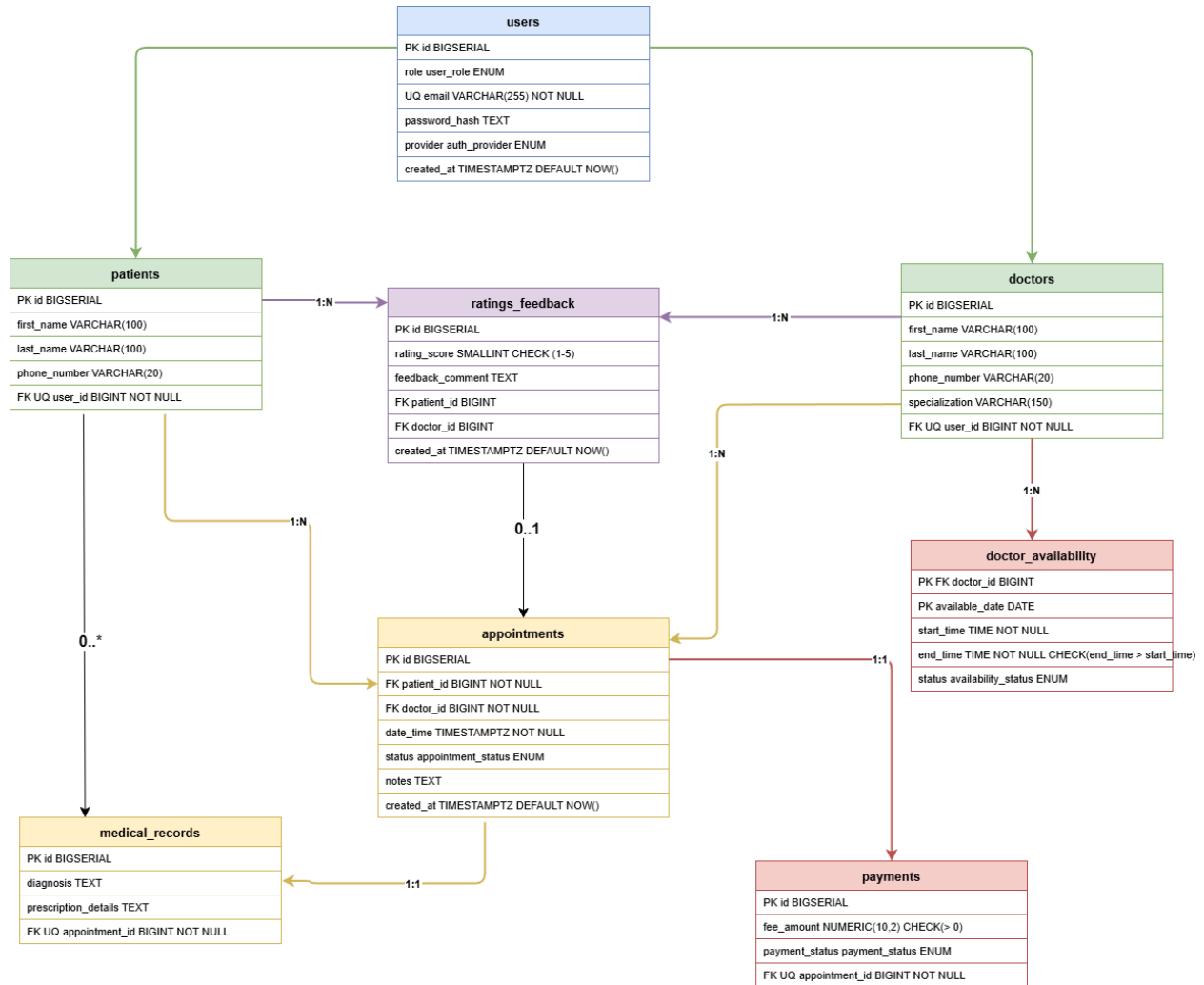
Status	Meaning
200 OK	Successful fetch or update
201 Created	Successful resource creation
204 No Content	Successful deletion
400 Bad Request	Validation errors, malformed JSON, illegal arguments
401 Unauthorized	Missing, invalid, or expired JWT; bad credentials
403 Forbidden	Insufficient role; or record locked pending payment
404 Not Found	Resource not found
409 Conflict	Duplicate email, duplicate medical record or payment for appointment, slot already booked
422 Unprocessable Entity	Non-duplicate DB integrity issues
500 Internal Server Error	Unexpected failures
501 Not Implemented	Google OAuth2 mobile token exchange (stub)

Common Error Codes:

1. AUTH-001 invalid credentials
2. AUTH-002 missing/invalid/expired JWT
3. AUTH-003 insufficient permissions or payment not PAID
4. VALID-001 request validation failed
5. APPT-001 appointment not found
6. APPT-002 invalid appointment state transition
7. APPT-003 slot not available or one-day-ahead rule violated
8. RECORD-001 duplicate medical record for appointment
9. PAY-001 duplicate payment for appointment
10. PAY-002 webhook signature verification failed
11. SYSTEM-001 unhandled server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



users

Stores authentication-level fields. doctors and patients link back here one-to-one.

Column	Type	Notes
id	BIGSERIAL PK	Primary key
email	VARCHAR(255) UQ NOT NULL	Unique login email
password_hash	TEXT	BCrypt hash; NULL for OAuth-only accounts
role	user_role ENUM	PATIENT DOCTOR
provider	auth_provider ENUM	LOCAL GOOGLE
profile_image_url	TEXT	Supabase-hosted profile photo URL
created_at	TIMESTAMP TZ DEFAULT NOW ()	Account creation timestamp

patients

Column	Type	Notes
id	BIGSERIAL PK	Primary key
user_id	BIGINT FK UQ NOT NULL	One-to-one → users.id
first_name	VARCHAR(100)	
last_name	VARCHAR(100)	
phone_number	VARCHAR(20)	

doctors

Column	Type	Notes
id	BIGSERIAL PK	Primary key
user_id	BIGINT FK UQ NOT NULL	One-to-one → users.id

first_name	<code>VARCHAR(100)</code>	
last_name	<code>VARCHAR(100)</code>	
phone_number	<code>VARCHAR(20)</code>	
specializations	<code>VARCHAR(500)</code>	

specializations & doctor_specializations

Doctors use a many-to-many relationship to specializations.

Column	Type	Notes
id	<code>BIGSERIAL PK</code>	
name	<code>VARCHAR(100) UQ NOT NULL</code>	e.g. Cardiology, Dermatology

Column	Type	Notes
doctor_id	<code>BIGINT FK</code>	→ doctors.id
specialization_id	<code>BIGINT FK</code>	→ specializations.id

doctor_schedule_templates

Weekly recurring availability templates (one per doctor per day of week).

Column	Type	Notes
id	<code>BIGSERIAL PK</code>	
doctor_id	<code>BIGINT FK NOT NULL</code>	→ doctors.id
day_of_week	<code>day_of_week ENUM</code>	MONDAY ... SUNDAY
start_time	<code>TIME NOT NULL</code>	
end_time	<code>TIME NOT NULL</code>	CHECK(end_time > start_time)
is_available	<code>BOOLEAN DEFAULT TRUE</code>	False = doctor unavailable on this weekday

doctor_availability

Date-specific overrides. UNAVAILABLE removes unbooked slots; AVAILABLE uses custom hours.

Column	Type	Notes
id	<code>BIGSERIAL PK</code>	Surrogate key
doctor_id	<code>BIGINT FK NOT NULL</code>	→ doctors.id
available_date	<code>DATE NOT NULL</code>	UQ(doctor_id, available_date)

start_time	TIME	Used when status = AVAILABLE
end_time	TIME	CHECK(end_time > start_time)
status	availability_status ENUM	AVAILABLE UNAVAILABLE

appointment_slots

Auto-generated 30-minute slots (nightly job, 14 days ahead). Canonical source for booking times.

Column	Type	Notes
id	BIGSERIAL PK	
doctor_id	BIGINT FK NOT NULL	→ doctors.id
slot_date	DATE NOT NULL	
start_time	TIME NOT NULL	
end_time	TIME NOT NULL	Always start_time + 30 minutes
status	slot_status ENUM	AVAILABLE BOOKED

appointments

Linked to patient, doctor, and optionally a generated slot. Legacy date/time/date_time fields retained for compatibility.

Column	Type	Notes
id	BIGSERIAL PK	
patient_id	BIGINT FK NOT NULL	→ patients.id
doctor_id	BIGINT FK NOT NULL	→ doctors.id
slot_id	BIGINT FK	→ appointment_slots.id (canonical)
date	DATE	Legacy — retained for compatibility
time	TIME	Legacy — retained for compatibility
date_time	TIMESTAMPTZ	Legacy — retained for compatibility
status	appointment_status ENUM	BOOKED COMPLETED CANCELED
notes	TEXT	Reason for visit
created_at	TIMESTAMPTZ DEFAULT NOW ()	

medical_records

Per-appointment diagnosis and prescription. Now includes full prescription fields.

Column	Type	Notes
id	BIGSERIAL PK	
appointment_id	BIGINT FK UQ NOT NULL	One-to-one → appointments.id
diagnosis	TEXT	
medicine_name	VARCHAR(255)	Used for OpenFDA lookup
dosage	VARCHAR(100)	
route	VARCHAR(100)	e.g. oral, IV
frequency	VARCHAR(100)	e.g. twice daily
duration	VARCHAR(100)	e.g. 7 days
prescription_notes	TEXT	Additional prescription instructions

medical_record_files

Primary file storage table. Files uploaded to Supabase Storage.

Column	Type	Notes
id	BIGSERIAL PK	
uploaded_by_user	BIGINT FK NOT NULL	→ users.id
patient_id	BIGINT FK NOT NULL	→ patients.id
appointment_id	BIGINT FK	Optional → appointments.id
storage_path	TEXT NOT NULL	Supabase Storage object path
file_url	TEXT NOT NULL	Supabase signed URL base
file_name	VARCHAR(255)	Original filename
file_type	VARCHAR(50)	MIME type
created_at	TIMESTAMPTZ DEFAULT NOW ()	

Note: a legacy file_uploads table still exists and is used by older /api/files and /api/record-files routes retained for backward compatibility.

payments

One-to-one with appointments. Stores PayMongo-specific identifiers.

Column	Type	Notes
id	BIGSERIAL PK	
appointment_id	BIGINT FK UQ NOT NULL	One-to-one → appointments.id

<code>fee_amount</code>	<code>NUMERIC(10,2)</code> <code>CHECK(> 0)</code>	Reservation fee or total bill
<code>payment_status</code>	<code>payment_status</code> <code>ENUM</code>	PENDING PAID FAILED REFUNDED
<code>paymongo_session_id</code>	<code>VARCHAR(255)</code>	PayMongo checkout session identifier
<code>paymongo_payment_intent_id</code>	<code>VARCHAR(255)</code>	PayMongo payment intent identifier
<code>paid_at</code>	<code>TIMESTAMP TZ</code>	Set when webhook confirms PAID

ratings_feedback

One per appointment. Points to both the patient and doctor involved.

Column	Type	Notes
<code>id</code>	<code>BIGSERIAL PK</code>	
<code>appointment_id</code>	<code>BIGINT FK UQ NOT NULL</code>	One-to-one → appointments.id
<code>patient_id</code>	<code>BIGINT FK NOT NULL</code>	→ patients.id
<code>doctor_id</code>	<code>BIGINT FK NOT NULL</code>	→ doctors.id
<code>rating_score</code>	<code>SMALLINT CHECK(1-5)</code>	1 to 5 stars
<code>feedback_comment</code>	<code>TEXT</code>	Optional; anonymous supported
<code>created_at</code>	<code>TIMESTAMP TZ DEFAULT NOW()</code>	

refresh_tokens

Column	Type	Notes
<code>id</code>	<code>BIGSERIAL PK</code>	
<code>user_id</code>	<code>BIGINT FK NOT NULL</code>	→ users.id
<code>token</code>	<code>TEXT NOT NULL</code>	Refresh token value
<code>expiry_date</code>	<code>TIMESTAMP TZ NOT NULL</code>	

o Key Relationships

Relationship	Description
users ↔ patients / doctors	One-to-one (supertype/subtype). Every user is either a patient or a doctor.
doctors ↔ specializations	Many-to-many via doctor_specializations join table; denormalized specializations_summary column on doctors for reporting.

doctors → doctor_schedule_templates	One-to-many. Each doctor can have one template per day of week.
doctors → doctor_availability	One-to-many. Date-specific overrides with unique constraint on (doctor_id, available_date).
doctors → appointment_slots	One-to-many. Slots are auto-generated from templates and availability overrides.
patients → appointments	One-to-many. A patient books many appointments.
doctors → appointments	One-to-many. A doctor has many appointments.
appointments → appointment_slots	Many-to-one. Slot is the canonical source of booking time.
appointments → medical_records	One-to-one (optional).
appointments → payments	One-to-one.
appointments → ratings_feedback	One-to-one (optional).
appointments → medical_record_files	One-to-many (optional). Multiple files per appointment.

7.0 UI/UX DESIGN

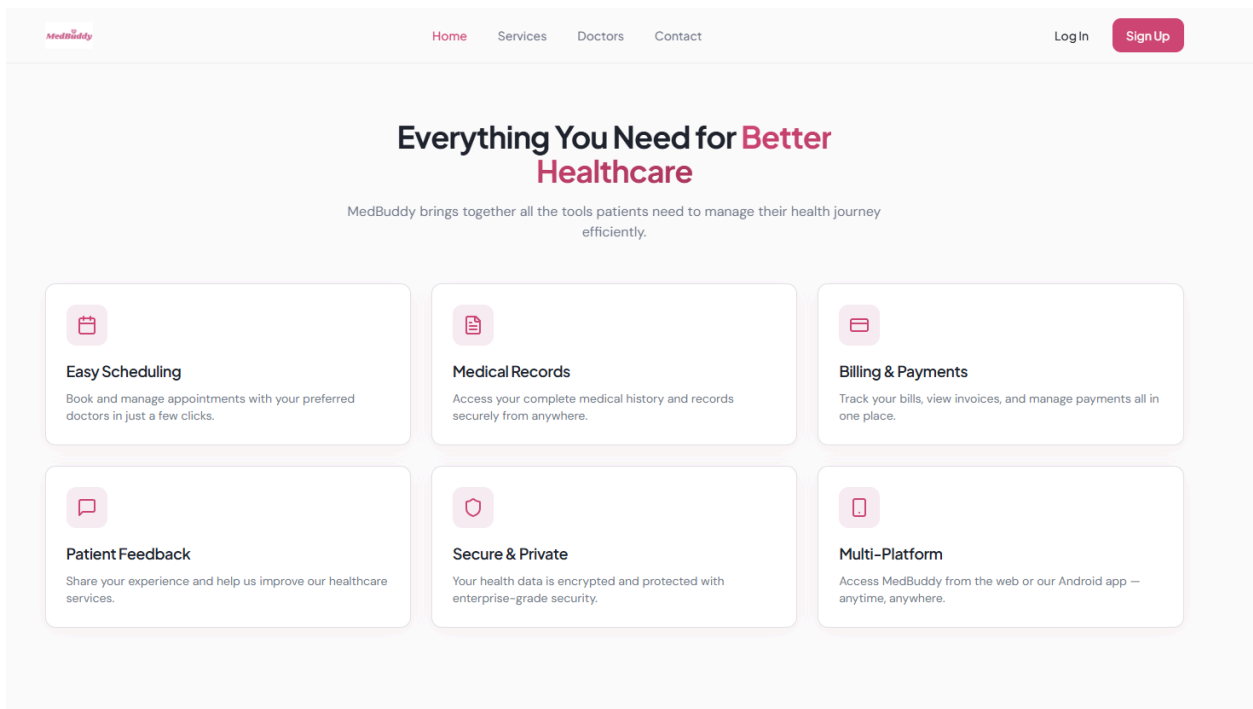
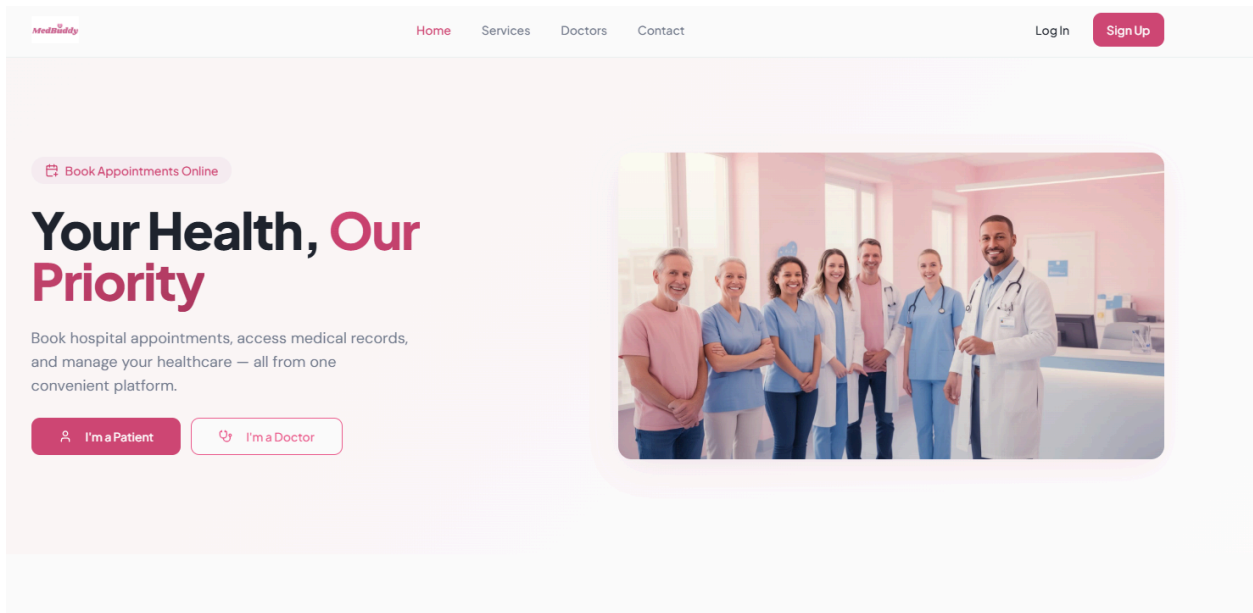
7.1 Web Application Wireframes

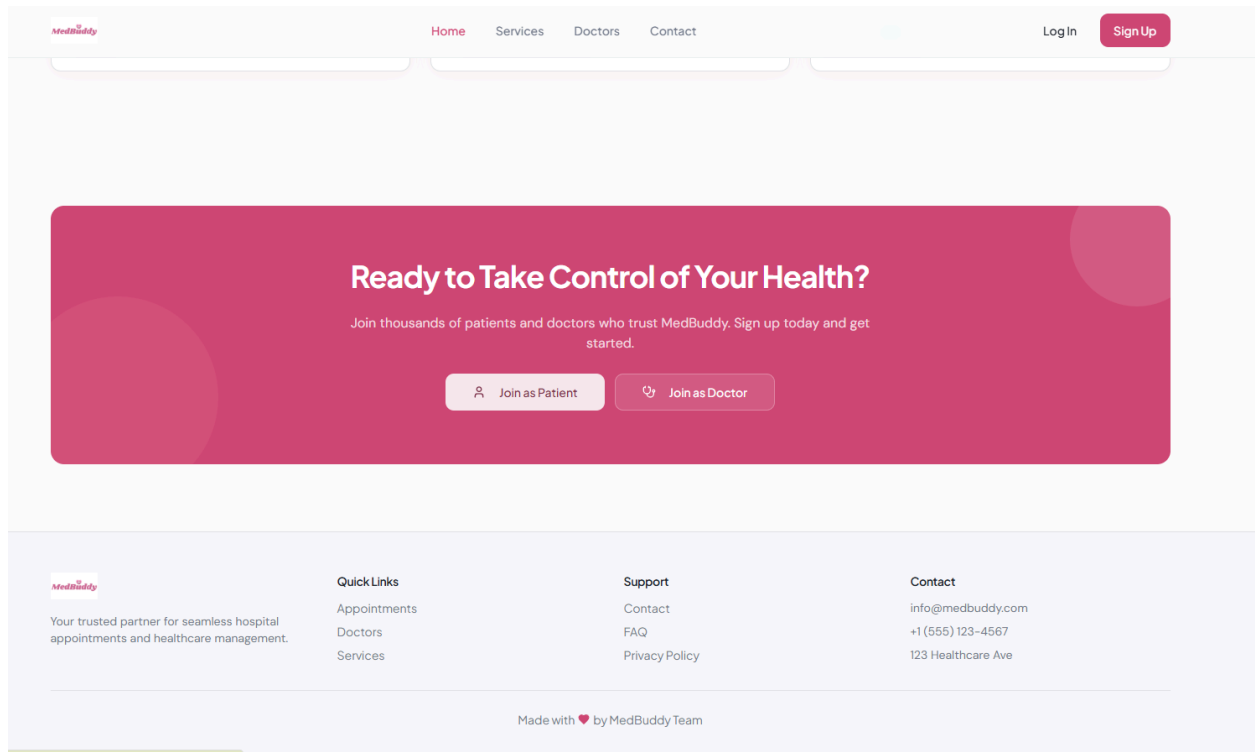
Landing Page

Header: MedBuddy logo, Login, Register, CTA (Book Appointment)

Hero: value proposition, short steps, doctor/patient entry points

Footer: contact, about, privacy/terms

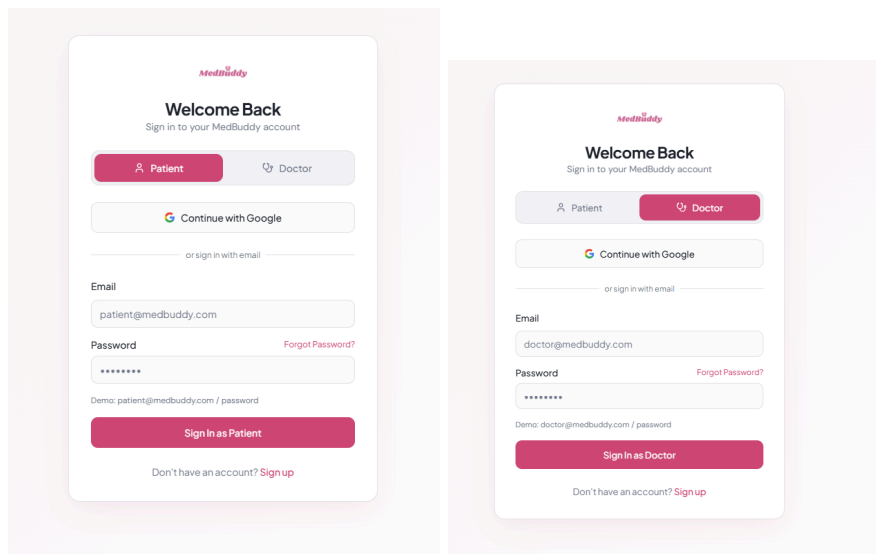




Authentication

Login Page: email, password, Google login, "Forgot Password"

Register Page: first/last name, email, password, role (Patient/Doctor), submit



The image displays two wireframe designs for a 'Create Account' form in the MedBuddy app. Both designs feature the MedBuddy logo at the top, followed by the title 'Create Account' and the subtitle 'Join MedBuddy to manage your healthcare'.

Left Design (Doctor Account):

- Role selection: Two buttons, 'Patient' (light blue) and 'Doctor' (dark blue), with the 'Doctor' button selected.
- First Name: Input field with 'John'.
- Last Name: Input field with 'Doe'.
- Email: Input field with 'john@example.com'.
- Password: Input field with masked characters '.....'.
- Role: Dropdown menu with 'Doctor' selected.
- Specialization: Dropdown menu with 'Select specialization'.
- Create Account: Large dark blue button.
- Footer: Link 'Already have an account? [Sign in](#)'.

Right Design (Patient Account):

- Role selection: Two buttons, 'Patient' (dark blue) and 'Doctor' (light blue), with the 'Patient' button selected.
- First Name: Input field with 'John'.
- Last Name: Input field with 'Doe'.
- Email: Input field with 'john@example.com'.
- Password: Input field with masked characters '.....'.
- Role: Dropdown menu with 'Patient' selected.
- Create Account: Large dark blue button.
- Footer: Link 'Already have an account? [Sign in](#)'.

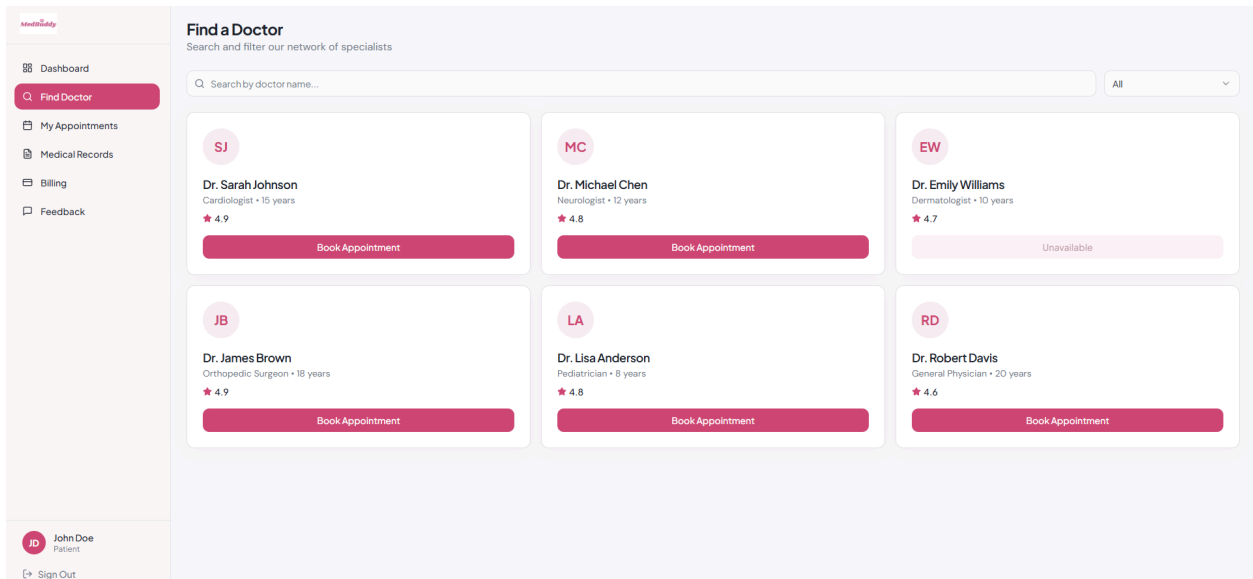
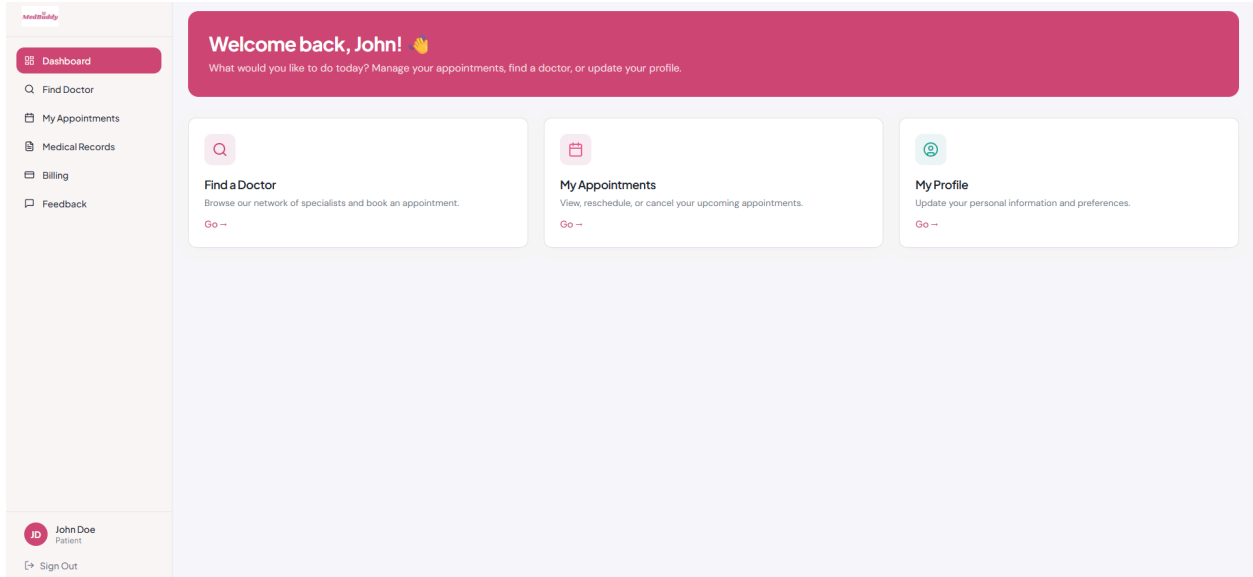
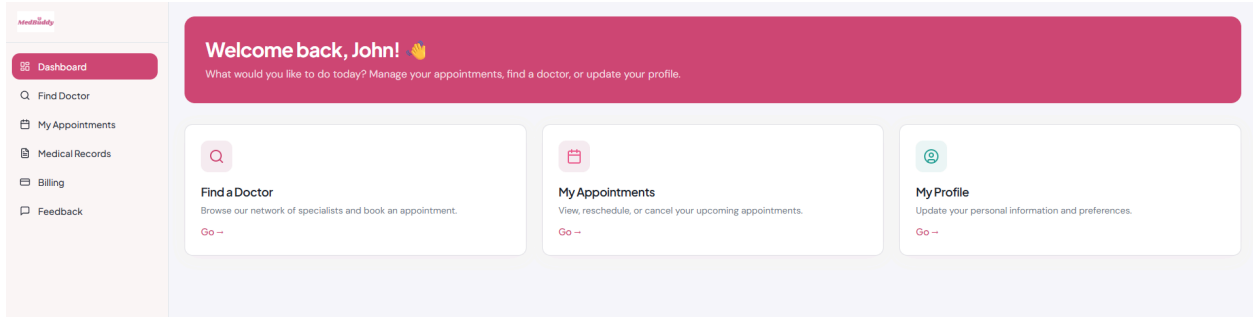
Patient Flow

Patient Dashboard: welcome header, quick cards (Find Doctor, My Appointments, Profile)

Find Doctor Page: search/filter (name, specialization), doctor list cards, Book button

Book Appointment Page: selected doctor info, date-time picker, notes, Confirm Booking

My Appointments Page: grouped by status (Pending, Confirmed, Completed, Cancelled), reschedule/cancel actions



← Back

Book Appointment

Confirm your appointment details below

SJ

Dr. Sarah Johnson

Cardiologist • 15 years

★ 4.9

Select Date

<

March 2026

>

Su

Mo

Tu

We

Th

Fr

Sa

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

1

2

3

4

Select Time Slot

9:00 AM

10:00 AM

11:00 AM

2:00 PM

3:00 PM

Notes (optional)

Any symptoms or notes for the doctor...

Confirm Booking

medilife Dashboard Find Doctor My Appointments Medical Records Billing Feedback JD John Doe Patient Sign Out	My Appointments View and manage your appointments Pending2 MC Dr. Michael Chen Neurologist Mar 12, 2026 2:30 PM Pending Reschedule Cancel RD Dr. Robert Davis General Physician Mar 18, 2026 3:00 PM Pending Reschedule Cancel Confirmed1 SJ Dr. Sarah Johnson Cardiologist Mar 5, 2026 10:00 AM Confirmed Reschedule Cancel Completed1 JB Dr. James Brown Orthopedic Surgeon Feb 20, 2026 9:00 AM Completed Cancelled1 LA Dr. Lisa Anderson Pediatrician Feb 10, 2026 11:00 AM Cancelled
--	--

medallia

Dashboard

Find Doctor

My Appointments

Medical Records

Billing

Feedback

JD

John Doe

Patient

Sign Out

Medical Records

Access and download your health documents

Q Search records...

All

Complete Blood Count

Lab Results • Dr. Sarah Johnson • Feb 20, 2026

Blood Pressure Medication

Prescription • Dr. Michael Chen • Feb 15, 2026

Chest X-Ray Report

Imaging • Dr. James Brown • Jan 28, 2026

Lipid Panel

Lab Results • Dr. Sarah Johnson • Jan 10, 2026

Neurology Follow-up

Consultation • Dr. Michael Chen • Dec 18, 2025

Allergy Medication

Prescription • Dr. Emily Williams • Dec 5, 2025

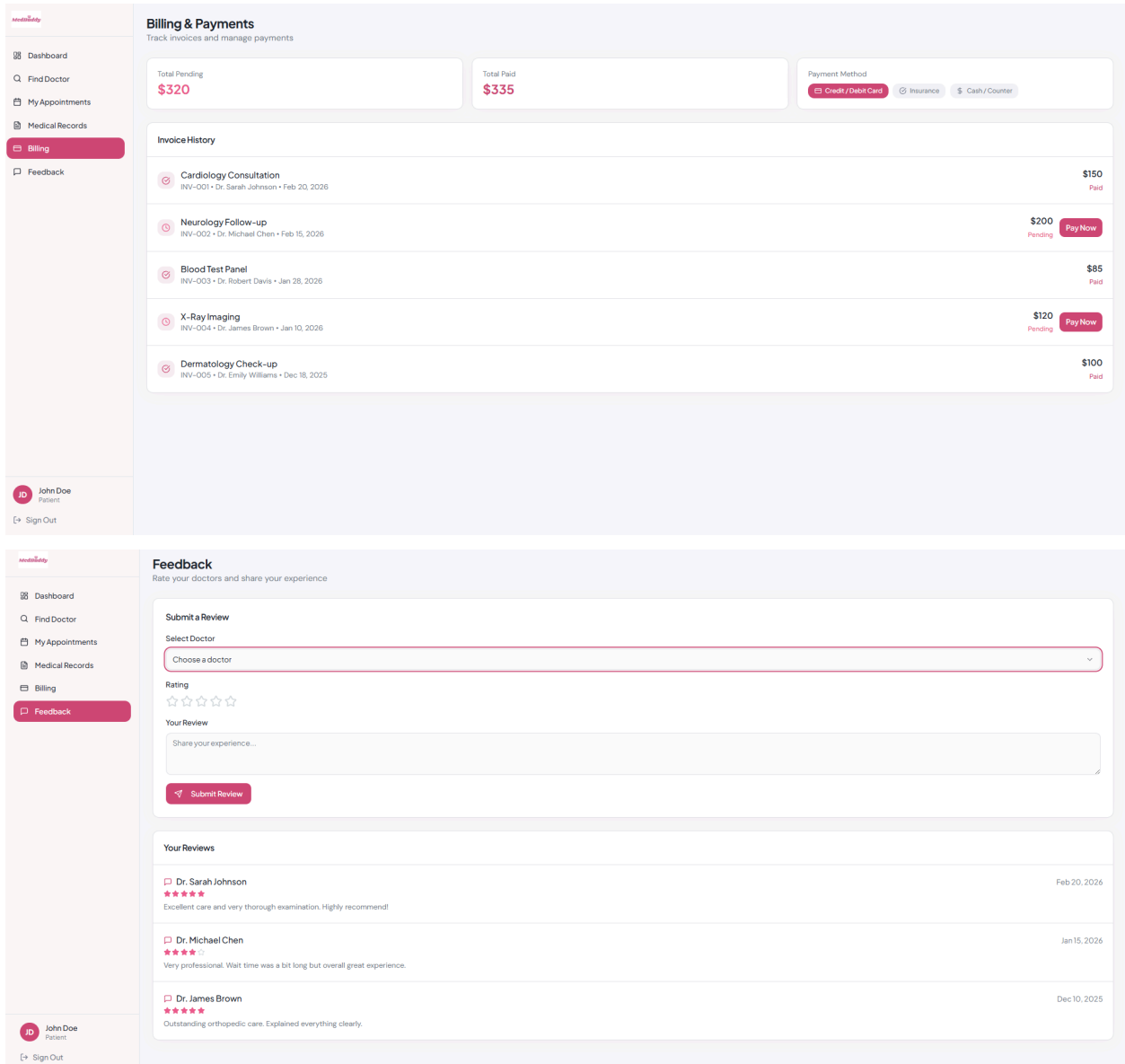
HbA1c Test

Lab Results • Dr. Robert Davis • Nov 22, 2025

MRI Brain Scan

Imaging • Dr. Michael Chen • Nov 10, 2025

44



Doctor Flow

Doctor Dashboard: today's schedule summary, quick links, profile/availability shortcut

Doctor Appointments Page: appointment list by status/date, confirm/reject/complete actions, patient notes preview

mediality

Dashboard

Appointments

Schedule

Patient Records

Settings

SJ Sarah Johnson
Doctor

Sign Out

Good morning, Dr. Johnson! 🙌

You have 4 appointments scheduled for today.

Manage Appointments

Patient Records

Profile & Availability

Today's Schedule

View all

JD John Doe
Follow-up

🕒 9:00 AM
Checked In

JS Jane Smith
New Patient

🕒 10:00 AM
Waiting

MW Mike Wilson
Consultation

🕒 11:30 AM
Scheduled

ED Emily Davis
Follow-up

🕒 2:00 PM
Scheduled

mediality

Dashboard

Appointments

Schedule

Patient Records

Settings

SJ Sarah Johnson
Doctor

Sign Out

Appointment Management

View and manage all patient appointments

Q Search patients...

All Status

JD John Doe
Follow-up • Mar 5, 2026 at 9:00 AM

Confirmed Complete

Blood pressure check

JS Jane Smith
New Patient • Mar 5, 2026 at 10:00 AM

Pending Confirm Reject

First visit, chest pain

MW Mike Wilson
Consultation • Mar 6, 2026 at 11:30 AM

Confirmed Complete

Headaches for 2 weeks

ED Emily Davis
Follow-up • Mar 7, 2026 at 2:00 PM

Pending Confirm Reject

Post-surgery review

TH Tom Harris
Check-up • Mar 8, 2026 at 9:30 AM

Rejected

SM Sarah Miller
Consultation • Mar 3, 2026 at 3:00 PM

Completed

Annual checkup done

medibuddy

Dashboard

Appointments

Schedule

Patient Records

Settings

SI

Sarah Johnson

Doctor

Sign Out

Schedule Management

Set your weekly availability and time slots

Save Changes

Weekly Availability

Monday

09:00 AM

to

05:00 PM

30min slots

Tuesday

09:00 AM

to

05:00 PM

30min slots

Wednesday

09:00 AM

to

05:00 PM

30min slots

Thursday

09:00 AM

to

05:00 PM

30min slots

Friday

09:00 AM

to

05:00 PM

30min slots

Saturday

Unavailable

Sunday

Unavailable

Holidays & Days Off

mm/dd/yyyy

+ Add Holiday

2026-03-15

2026-04-20

medibuddy

Dashboard

Appointments

Schedule

Patient Records

Settings

SI

Sarah Johnson

Doctor

Sign Out

Patient Records

Browse and manage patient information

Q Search patients...

JD

John Doe

45y • Male • 8 visits

JS

Jane Smith

32y • Female • 3 visits

MW

Mike Wilson

68y • Male • 12 visits

ED

Emily Davis

28y • Female • 5 visits

TH

Tom Harris

40y • Male • 10 visits

SM

Sarah Miller

42y • Female • 6 visits

Select a patient to view details

medibuddy

Dashboard

Appointments

Schedule

Patient Records

Settings

SI

Sarah Johnson

Doctor

Sign Out

Settings

Manage your profile and preferences

Profile Information

First Name

Sarah

Last Name

Johnson

Email

doctor@medibuddy.com

Phone

+1 555-0201

Specialization

Cardiologist

Security

Current Password

New Password

Notifications

Email Notifications

Receive updates via email

SMS Notifications

Receive updates via SMS

Appointment Reminders

Get notified about upcoming appointments

47

7.2 Mobile Application Wireframes

Patient Bottom Nav: Dashboard | Find Doctor | Appointments | Profile

Doctor Bottom Nav: Dashboard | Appointments | Availability | Profile

Login, Register

The image displays two mobile application wireframes for the MedBuddy app. The left wireframe is the Login screen, and the right wireframe is the Register screen. Both screens feature the MedBuddy logo at the top, which consists of the word 'MedBuddy' in a pink, rounded font with a small heart icon above the 'y'.

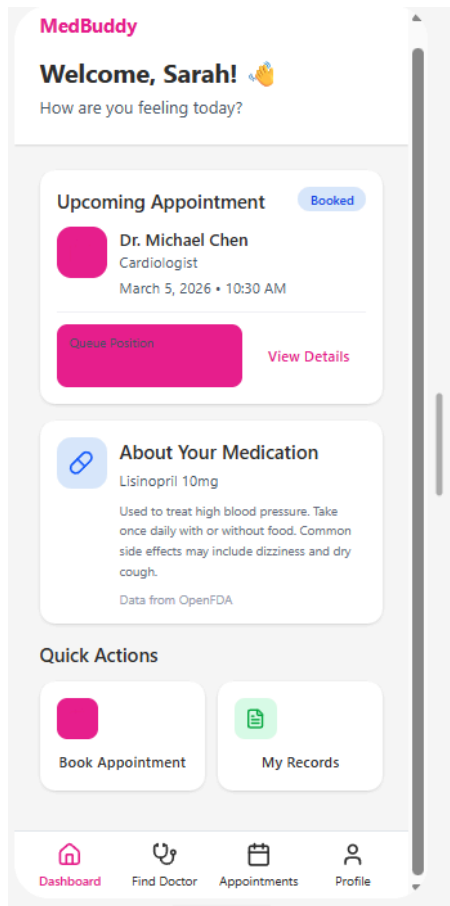
Login Screen (Left):

- Fields for 'Email' and 'Password' with placeholder text 'Enter your email' and 'Enter your password' respectively.
- A pink 'Log In' button.
- A link 'Or continue with' followed by a 'Sign in with Google' button featuring the Google logo.
- A link 'Don't have an account? Sign Up'.

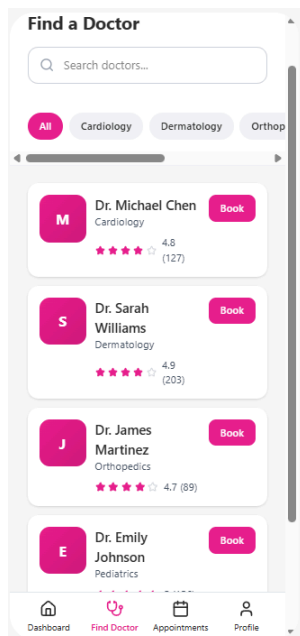
Register Screen (Right):

- Section title 'Create Account'.
- Fields for 'First Name' (containing 'John') and 'Last Name' (containing 'Doe').
- Fields for 'Email' (containing 'john.doe@example.com') and 'Password' (containing '*****').
- A 'Confirm Password' field (containing '*****').
- A section 'I'm a...' with two buttons: 'Patient' (pink) and 'Doctor' (white).
- A pink 'Create Account' button.
- A link 'Already have an account? Log In'.

Patient Dashboard



Find Doctor (search + doctor cards)



Book Appointment (date/time + notes + confirm button)

Book Appointment

C

Dr. Michael Chen

Cardiologist

★ 4.8 (127 reviews)

Select Date

Mar 3Mar 4Mar 5

Select Time

09:00 AM

09:30 AM

10:00 AM

10:30 AM

11:00 AM

11:30 AM

02:00 PM

02:30 PM

03:00 PM

03:30 PM

04:00 PM

04:30 PM

Reason for Visit

Describe your symptoms or reason for visit...

Confirm Booking

My Appointments (status tabs/chips + item actions)

My Appointments

AllBookedCompletedCanceled

M

Dr. Michael Chen

Cardiology

March 5, 2026 • 10:30 AM

Cancel

Reschedule

S

Dr. Sarah Williams

Dermatology

February 28, 2026 • 2:00 PM

View Record

J

Dr. James Martinez

Orthopedics

February 15, 2026 • 11:00 AM

Dashboard

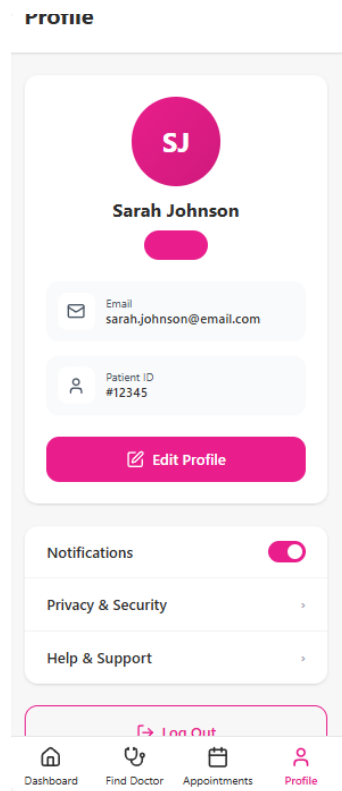
Find Doctor

Appointments

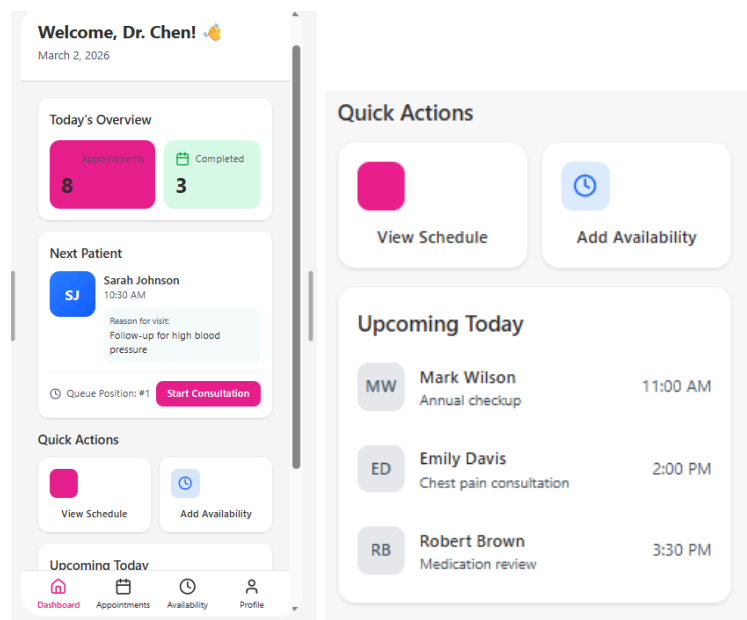
Profile

50

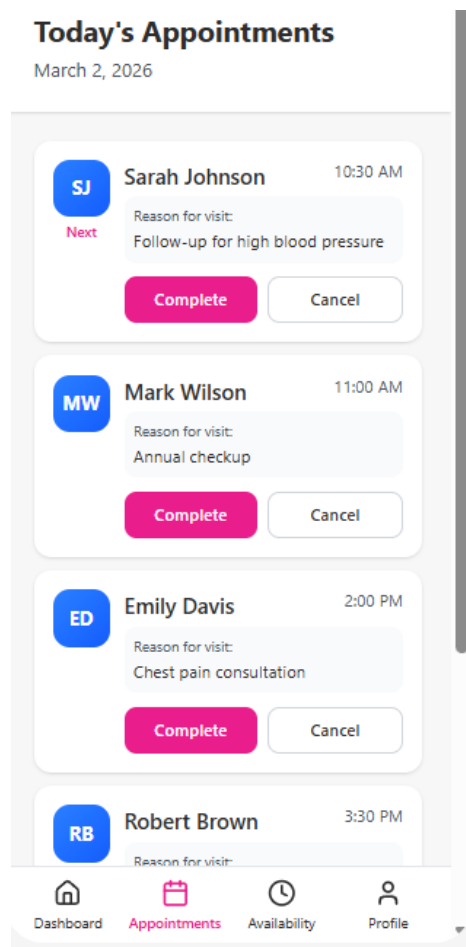
Profile Page (status tabs/chips + item actions)



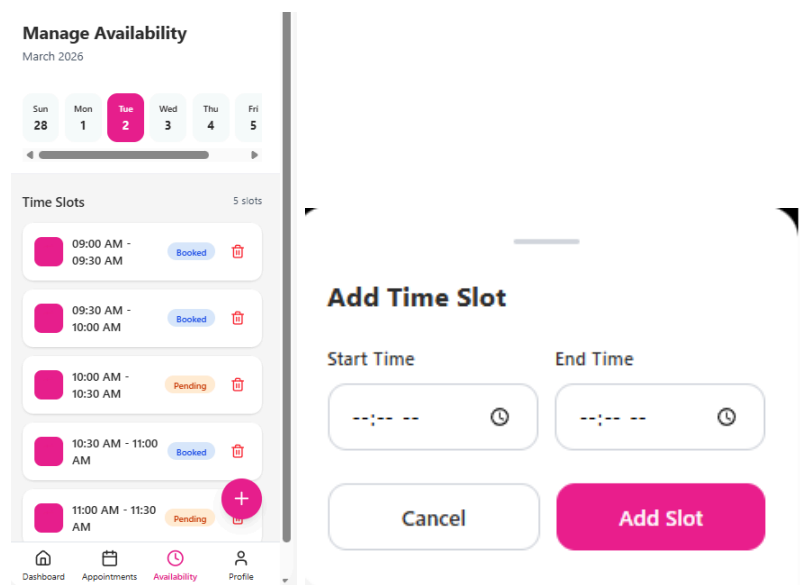
Doctor Dashboard



Doctor Appointment Management (confirm/reject/complete)

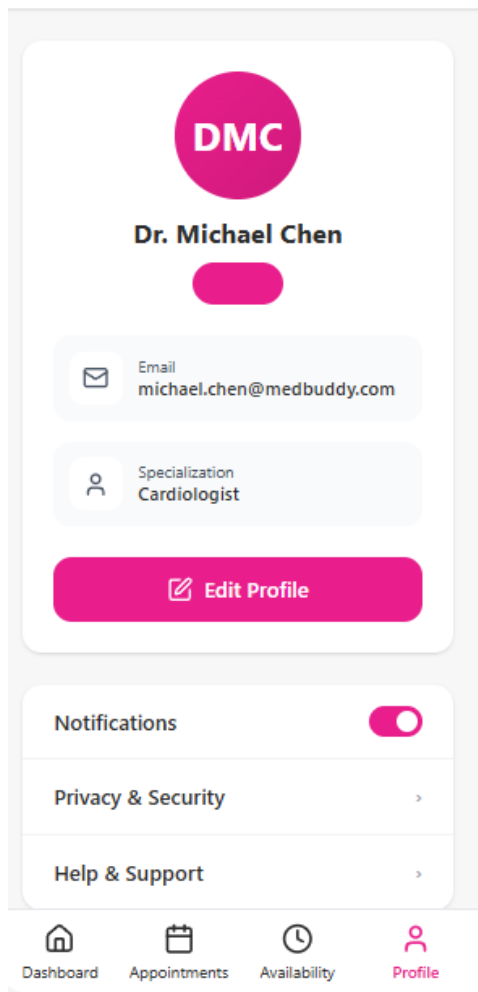


Availability



Profile Page

Profile



Mobile-Specific Features:

- Touch-optimized buttons and tap targets (minimum 48x48dp per Android Material Design guidelines)
- Gesture support: swipe left on appointment cards to reveal Cancel action; pull-to-refresh on Appointments, Find Doctor, and Medical Records screens
- Offline caching for doctor profile data and recent appointments using Android's local storage

- Bottom navigation bar for all authenticated screens — Patient (Dashboard, Find Doctor, Appointments, Profile) and Doctor (Dashboard, Appointments, Availability, Profile)
- Simplified forms for mobile input — date picker uses Android native DatePickerDialog; time slot selection uses a grid tap UI instead of dropdowns; text fields use appropriate keyboard types (email, phone, text)

Design System:

- **Colors:**
 - Primary: #E91E8C (MedBuddy pink — buttons, active nav, highlights)
 - Primary Dark: #C2185B (pressed/hover state of primary)
 - Background: #F7F7F7 (screen backgrounds)
 - Surface: #FFFFFF (cards, bottom nav, input fields)
 - Success: #10B981 (Completed status, Paid status)
 - Warning: #F59E0B (Pending payment status)
 - Error: #EF4444 (Canceled status, validation errors)
 - Text Primary: #2D2D2D
 - Text Secondary: #888888
 - Border / Divider: #E0E0E0
 - Pink Light: #FDE8F5 (chip backgrounds, subtle highlights)
- **Typography:**
 - Font family: Inter (or system default — Roboto on Android)
 - Heading 1: 24sp bold (screen titles)
 - Heading 2: 18sp semi-bold (section headers, card titles)
 - Body: 14sp regular (general content)
 - Caption: 12sp regular (timestamps, secondary labels)
 - Button: 14sp semi-bold, all caps optional
- **Spacing:** 8px grid system
- **Components:**

- Buttons: Primary (pink filled, white text, 8dp radius, 48dp height), Secondary (pink outlined, pink text), Destructive (red outlined for Cancel actions)
- Inputs: White background, 1dp gray border, 8dp radius, 48dp height, pink focused border, red error border with message below
- Cards: White background, 12dp radius, 2dp elevation shadow, 16dp internal padding
- Status Chips: 6dp radius, colored background at 15% opacity with matching text color — Booked (blue), Completed (green), Canceled (gray), Pending (orange), Paid (green)
- Bottom Nav: White background, 2dp top border, active icon and label in pink, inactive in gray
- Skeleton Loaders: Gray shimmer blocks matching the shape of the content they replace — used on all screens during API fetch
- Modals / Dialogs: Use Android AlertDialog style — white background, pink confirm button, gray cancel button, 12dp radius
- **Responsive:**
 - Mobile-first — all layouts designed for 360dp minimum width
 - Tablet adaptation (600dp+): two-column layout for doctor list and appointment list
 - Font scaling: respect Android system font size settings
 - All touch targets minimum 48x48dp throughout

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1–2: Project setup — initialize repository structure (IT342_MedBuddy_G2_Aquino), set up Spring Boot project via Spring Initializr, create React app, create Android project in Android Studio

Day 3–4: Complete FRS, NFR, MoSCoW feature list, and acceptance criteria

Day 5–7: Finalize system architecture diagram, component diagram, and layered architecture documentation

Week 2: Detailed Design

Day 1–2: Complete API endpoint reference (all controllers, methods, request/response DTOs)

Day 3–4: Finalize database schema — all 8 PostgreSQL tables, relationships, constraints

Day 5–6: UI/UX wireframes for web (React) and mobile (Android XML layouts) — patient and doctor flows

Day 7: Implementation plan finalization; set up Neon database, Render project, and Vercel project

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot configuration — application.properties, Neon PostgreSQL JDBC connection, environment variables on Render

Day 2: JPA entities and repositories — User, Patient, Doctor, DoctorAvailability

Day 3: JWT implementation — JwtUtil, JwtAuthFilter, SecurityConfig, BCrypt password hashing

Day 4: Auth endpoints — POST /api/auth/register, POST /api/auth/login, GET /api/auth/me; welcome email via SMTP

Day 5: Google OAuth2 login — Spring Security OAuth2 config, custom JWT issuance after Google login

Week 4: Core Features

Day 1: Appointment CRUD — entities, repository, service, controller; full validation (no past dates, no double booking)

Day 2: Doctor availability management — CRUD endpoints; slot status transitions

Day 3: Medical records — create/read endpoints; file upload (multipart) linked to records

Day 4: Payments — Payment entity, PayMongo sandbox integration, webhook handler, receipt email

Day 5: Feedback, OpenFDA external API integration (DrugInfoService via RestTemplate), global exception handler, ApiResponse envelope, API testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup — Vite, Tailwind CSS, Axios instance with JWT interceptor, AuthContext, ProtectedRoute

Day 2: Authentication pages — Register, Login, Google OAuth2 button; JWT storage in localStorage

Day 3: Doctor listing and availability calendar — browse doctors, view available slots

Day 4: Appointment booking flow — select doctor, date, time, reason; confirmation screen

Day 5: Patient dashboard — upcoming appointments, queue status (polling every 10 seconds)

Week 6: Complete Web Features

Day 1: Medical records view — patient history, diagnosis, prescription, file download, "About Your Medication" card from OpenFDA

Day 2: Payment flow — fee display, PayMongo sandbox payment UI, success/failure handling

Day 3: Doctor dashboard — today's appointments, mark complete/cancel, create medical record, manage availability

Day 4: Feedback submission form; doctor profile with average rating; responsive design polish

Day 5: Full web-to-API integration testing across all features; fix bugs; deploy to Vercel

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio project setup — XML layouts only, Retrofit 2 instance, OkHttp JWT interceptor, EncryptedSharedPreferences helper

Day 2: LoginActivity and RegisterActivity — connect to /api/auth/register and /api/auth/login; store JWT; Google Sign-In SDK

Day 3: Role-based navigation — PatientDashboardActivity and DoctorDashboardActivity based on JWT role claim

Day 4: Doctor list screen and availability calendar — RecyclerView, connect to /api/doctors and /api/doctors/{id}/availability

Day 5: Appointment booking screens — slot selection, reason input, confirm booking via POST /api/appointments

Week 8: Complete Mobile App

Day 1: Patient appointments list and queue status screen — polling /api/appointments/queue

Day 2: Medical records screen — diagnosis, prescription, file download, OpenFDA drug info card

Day 3: Payment status screen; feedback submission screen; doctor schedule and availability management screens

Day 4: UI polish — empty states, loading indicators, error messages; test on emulator and physical device (API 34)

Day 5: APK generation; full mobile-to-API integration testing; fix bugs

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across all three platforms — same user account on web and Android, data consistent

Day 2: Bug fixes — priority on auth flow, appointment booking, file upload, and payment webhook

Day 3: Security review — JWT expiration, CORS config, role enforcement, env vars not in source code

Day 4: Performance checks — API response times, Neon query performance, Render cold start behavior

Day 5: Documentation updates — finalize all docs in /docs folder; update README.md

Week 10: Deployment

Day 1: Backend final deployment to Render.com — verify all environment variables, test live API

Day 2: Web app final deployment to Vercel — verify REACT_APP_API_URL points to live Render URL

Day 3: Android APK final build — verify Retrofit BASE_URL points to live Render URL; distribute APK

Day 4: Full system smoke test — register, book, complete appointment, upload file, pay, submit feedback on all platforms

Day 5: Final documentation review and project submission

Milestones:

- **M1 (End Week 2):** All design documents complete — architecture, database schema, API reference, wireframes
- **M2 (End Week 4):** Spring Boot API fully functional and tested — all endpoints working, deployed to Render
- **M3 (End Week 6):** React web application complete and deployed to Vercel
- **M4 (End Week 8):** Kotlin Android app complete — APK builds and connects to live API
- **M5 (End Week 10):** Full system integrated, deployed, and submitted

Critical Path:

1. Neon + Render environment setup (Week 2) — everything else depends on a working database connection

2. JWT authentication system (Week 3) — blocks all protected endpoints on web and Android
3. Appointment booking module (Week 4) — the core business module; medical records, payments, and feedback all depend on it
4. Google OAuth2 + file upload + OpenFDA integration (Week 4–5) — mandatory IT342 integration requirements
5. Payment gateway sandbox (Week 4) — needs early setup as PayMongo/PayPal sandbox approval can take time
6. Cross-platform integration testing (Week 9) — validates that React and Android behave identically against the live API

Risk Mitigation:

- Set up Neon, Render, and Vercel accounts in Week 1 — deployment issues found late are the biggest risk for a solo project
- Register for PayMongo or PayPal sandbox early — approval and API key access can take a few days
- Build and test the Android app against the live Render API from Week 7 onwards — do not test only against localhost
- Commit working code to GitHub at the end of every day — never lose a working state
- Implement polling for queue status first; only upgrade to WebSocket if time permits in Week 9
- Scope down if behind schedule — payments and real-time queue are bonus/secondary features; auth, appointments, medical records, and file upload are non-negotiable for the final defense